

LAPORAN PRAKTIKUM PBO

Interaksi Objek & Manipulasi State (Mini Bank)

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

NURFIAH, S.ST, M.Kom

Asisten Pratikum:

Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

Langkah Pengerjaan:

A. Praktikum

1. Class Rekening

```
package pekan1;
import java.text.NumberFormat;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening (String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal >= 10000) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari
10000!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal: Minimal tarik 10000!");
        } else if (nominal > saldo) {
            System.out.println("Gagal : Saldo anda tidak cukup!");
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo anda saat ini: " + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : Rp" + saldo);
        System.out.println("-----");
    }
}
```

Class Rekening berfungsi sebagai model/blueprint (cetakan data) yang merepresentasikan sebuah rekening bank dalam sistem ini. Class ini bertugas

mendefinisikan data apa saja yang dimiliki sebuah rekening (nomor rekening, nama pemilik, saldo) serta perilaku/aksi apa saja yang bisa dilakukan terhadap rekening tersebut (setor tunai, tarik tunai, cek informasi). Class ini sendiri tidak dijalankan langsung, melainkan dicetak menjadi objek nyata (misalnya saat nasabah membuka rekening baru), sehingga setiap nasabah memiliki datanya masing-masing, namun tetap menggunakan logika transaksi yang sama tanpa perlu menulis ulang kode.

a. public Rekening

```
public Rekening (String nomor, String nama, double saldoAwal) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
    System.out.println("Rekening atas nama " + namaPemilik + "  
berhasil dibuat dengan saldo Rp" + saldo);  
}
```

Method ini merupakan constructor yang dipanggil secara otomatis setiap kali objek Rekening baru dibuat menggunakan `new Rekening(...)`. Constructor ini bertugas menyimpan nilai-nilai yang diterima dari parameter ke dalam atribut objek, yaitu `nomorRekening`, `namaPemilik`, dan `saldo`, kemudian mencetak pesan konfirmasi bahwa rekening telah berhasil dibuat lengkap dengan saldo awalnya. Dengan kata lain, constructor ini berfungsi menginisialisasi data sebuah rekening begitu objeknya pertama kali dibentuk, sehingga setiap rekening baru langsung memiliki identitas dan saldo yang jelas sejak awal.

b. public void setorTunai

```
public void setorTunai(double nominal) {  
    if (nominal >= 10000) {  
        saldo += nominal;  
        System.out.println("Setor tunai Rp" + nominal + "  
berhasil. Saldo saat ini: Rp" + saldo);  
    } else {  
        System.out.println("Gagal: Nominal setor harus lebih dari  
10000!");  
    }  
}
```

Method ini menangani proses penyetoran uang ke dalam rekening. Di dalamnya, program akan memeriksa apakah nominal yang disetorkan lebih besar atau sama dengan 10.000. Jika kondisi ini terpenuhi, maka nilai saldo akan bertambah sesuai nominal yang disetor dan program akan mencetak saldo terbaru sebagai konfirmasi keberhasilan. Namun jika nominal yang dimasukkan kurang dari batas minimal tersebut, transaksi akan ditolak dan pesan kegagalan akan ditampilkan tanpa mengubah nilai saldo sama sekali. Validasi ini bertujuan memastikan hanya setoran dengan nominal yang wajar yang dapat diproses oleh sistem.

c. public void tarikTunai

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {
```

```

        System.out.println("Gagal: Minimal tarik 10000!");
    }else if(nominal > saldo){
        System.out.println(" Gagal : Saldo anda tidak cukup!");
    }else {
        saldo -= nominal;
        System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo anda saat ini: " + saldo);
    }
}

```

Method ini menangani proses penarikan uang dengan dua lapis validasi sebelum transaksi benar-benar diproses. Pertama, sistem akan mengecek apakah nominal penarikan berada di bawah 10.000; jika iya, penarikan langsung ditolak dengan pesan bahwa minimal tarik adalah 10.000. Kedua, jika nominal yang ingin ditarik lebih besar daripada saldo yang tersedia, sistem juga akan menolak transaksi tersebut dengan alasan saldo tidak mencukupi. Hanya apabila kedua pengecekan ini terlewati barulah saldo akan dikurangi sesuai nominal penarikan, dan saldo terbaru akan ditampilkan ke layar. Struktur validasi berlapis ini penting agar rekening tidak bisa ditarik melebihi kemampuannya atau dengan nominal yang terlalu kecil.

d. public void cekInformasi

```

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

```

Method ini tidak melakukan perubahan apa pun terhadap data (bersifat read-only), melainkan hanya menampilkan informasi rekening yang sedang aktif ke layar. Informasi yang ditampilkan meliputi nomor rekening, nama pemilik, dan saldo akhir, yang disusun rapi dengan garis pembatas di bagian atas dan bawah agar mudah dibaca oleh pengguna. Method ini biasanya dipanggil ketika pengguna ingin memastikan status terkini dari rekening yang sedang digunakan.

2. Class Main

```

package pekan1;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");

```

```

        System.out.println("4. Cek Informasi Rekening");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");

        int pilihan = input.nextInt();
        input.nextLine();

        switch (pilihan) {
            case 1:
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukan Saldo Awal: ");
                double saldo = input.nextDouble();

                akunAktif = new Rekening(no, nama, saldo);
                break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                } else {
                    System.out.print("Masukkan nominal setor: ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor);
                }
                break;

            case 3:
                System.out.println("Fitur ini akan dikerjakan sebagai Tugas Mandiri.");
                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;

            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima kasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid!");
        }
    }
    input.close();
}

```

Class Main berfungsi sebagai pengendali program dan titik masuk yang menjalankan seluruh alur interaksi dengan pengguna. Class ini bertugas menampilkan menu pilihan

di layar, membaca input yang diketik pengguna, menyimpan seluruh rekening yang pernah dibuat ke dalam sebuah daftar (ArrayList), serta memutuskan objek Rekening mana yang sedang aktif digunakan. Setiap kali pengguna memilih suatu menu, misalnya setor atau tarik tunai, Main tidak memproses transaksi itu sendiri, melainkan memanggil method yang sudah tersedia di class Rekening untuk melakukan pekerjaan sebenarnya.

a. public class Main

```
package pekan1;
import java.util.Scanner;

public class Main1 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
```

Method ini adalah titik masuk (entry point) program. Di awal method, dibuat objek Scanner untuk membaca input dari keyboard, variabel akunAktif bertipe Rekening yang sengaja diinisialisasi dengan nilai null karena belum ada rekening yang dibuat saat program baru dijalankan, serta variabel boolean isRunning yang menjadi saklar agar program terus berjalan dalam perulangan menu sampai pengguna memilih keluar. Perlu diperhatikan bahwa versi ini berbeda dari versi sebelumnya: di sini tidak ada ArrayList<Rekening>, sehingga program hanya bisa menangani satu rekening aktif pada satu waktu — setiap kali menu 1 dipilih, rekening lama akan otomatis tertimpa oleh rekening baru.

b. Switch-Case

1. Case 1 Buka Rekening Baru

```
case 1:

        System.out.print("Masukkan No Rekening: ");
        String no = input.nextLine();
        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();
        System.out.print("Masukan Saldo Awal: ");
        double saldo = input.nextDouble();

        akunAktif = new Rekening(no, nama, saldo);
```

```
break;
```

Bagian ini meminta pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal secara berurutan. Setelah semua data terkumpul, program membuat objek Rekening baru dengan memanggil constructor (new Rekening(no, nama, saldo)), lalu hasilnya langsung disimpan ke variabel akunAktif.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 5
Pilihan tidak valid!
```

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Endy
Masukan Saldo Awal: 100000
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0
```

2. Case 2 Setor Tunai

```
case 2:

        if (akunAktif == null) {
            System.out.println("Error: Mohon maaf, Anda
belum memiliki nomor rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;
```

Program terlebih dahulu memeriksa apakah akunAktif masih null. Jika iya, artinya belum ada rekening yang dibuat sehingga transaksi tidak bisa dilakukan dan pesan error ditampilkan. Jika sudah ada rekening aktif, program meminta nominal setor dari pengguna, lalu memanggil method setorTunai() milik objek akunAktif — seluruh validasi nominal (minimal setor) diproses di dalam class Rekening, bukan di Main.

Output:

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp110000.0

```

3. Case 3 Tarik Tunai

```

case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda
belum memiliki nomor rekening!");
    }else {
        System.out.println("Masukkan nominal tarik:
");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;

```

Alurnya serupa dengan setor tunai: program memastikan ada akun aktif terlebih dahulu, kemudian meminta nominal tarik dari pengguna dan memanggil method tarikTunai() milik akunAktif. Semua validasi terkait batas minimal dan kecukupan saldo diproses di dalam class Rekening, sehingga class Main hanya berperan meneruskan input pengguna ke method yang bersangkutan.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
9000
Gagal: Minimal tarik 10000!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
100000
Tarik tunai Rp100000.0 berhasil. Saldo anda saat ini: 10000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
Gagal : Saldo anda tidak cukup!

```

4. Case 4 Cek Informasi Rekening

```
case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum
membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;
```

Program memeriksa apakah akunAktif masih null terlebih dahulu. Jika belum ada rekening yang dibuka, pesan error ditampilkan. Jika sudah ada, method cekInformasi() milik akunAktif dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo akhir ke layar.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
Error: Anda belum membuka rekening!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 12345
Nama Pemilik : Endy
Saldo Akhir  : Rp10000.0
-----
```

5. Case 0 Keluar

```
case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;
```

Saat menu ini dipilih, variabel isRunning diubah menjadi false sehingga perulangan while akan berhenti pada pengecekan kondisi berikutnya, disertai pesan penutup sebagai tanda program selesai dijalankan.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

B. Tugas Praktikum

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nomonal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

Case 3 Tarik Tunai

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda
belum memiliki nomor rekening!");
    }else {
        System.out.println("Masukkan nominal tarik:
");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Alurnya serupa dengan setor tunai: program memastikan ada akun aktif terlebih dahulu, kemudian meminta nominal tarik dari pengguna dan memanggil method tarikTunai() milik akunAktif. Semua validasi terkait batas minimal dan kecukupan saldo diproses di dalam class Rekening, sehingga class Main hanya berperan meneruskan input pengguna ke method yang bersangkutan.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Error: Mohon maaf, Anda belum memiliki nomor rekening!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
9000
Gagal: Minimal tarik 10000!
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
100000
Tarik tunai Rp100000.0 berhasil. Saldo anda saat ini: 10000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
Gagal : Saldo anda tidak cukup!

```

2. Tantangan Multi-Akun

Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

Case 5 Ganti Akun

```

case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Error: Belum ada
rekening yang terdaftar!");
        } else {
            System.out.println("Masukkan nomor rekening
yang ingin diaktifkan: ");

            String nomorDicari = input.nextLine();

            Rekening ditemukan = null;
            for (Rekening r : daftarRekening) {
                if
(r.nomorRekening.equalsIgnoreCase(nomorDicari)) {
                    ditemukan = r;
                    break;
                }
            }

            if (ditemukan != null) {
                akunAktif = ditemukan;
                System.out.println("Berhasil berpindah
ke rekening atas nama " + akunAktif.namaPemilik);
            } else {
                System.out.println("Error: Nomor
rekening '" + nomorDicari + "' tidak ditemukan!");
            }
        }
        break;

```

Bagian ini memungkinkan pengguna berpindah ke rekening lain yang sudah terdaftar. Program pertama-tama memeriksa apakah daftarRekening masih kosong; jika kosong, pengguna diberi tahu bahwa belum ada rekening yang bisa dipilih. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program melakukan pencarian di dalam daftarRekening menggunakan perulangan for. Apabila rekening dengan nomor tersebut ditemukan, akunAktif akan diperbarui ke rekening itu disertai pesan konfirmasi; jika tidak ditemukan, pesan error akan ditampilkan kepada pengguna.

Output:

```
=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Error: Belum ada rekening yang terdaftar!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
1
Masukan No Rekening:
12345
Masukan Nama Pemilik:
Endy
Masukan Saldo Awal: 100000
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
1
Masukan No Rekening:
12345
Error: Nomor rekening 12345sudah terdaftar!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
1
Masukan No Rekening:
23456
Masukan Nama Pemilik:
Pardilian
Masukan Saldo Awal: 10000
Rekening atas nama Pardilian berhasil dibuat dengan saldo Rp10000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
12345
Berhasil berpindah ke rekening atas nama Endy

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
23456
Berhasil berpindah ke rekening atas nama Pardilian
```